JobOS 技术实现说明

1. 项目概述 (Project Overview)

JobOS 是一个面向非技术 HR 和小团队的智能招聘 Agent。它利用大语言模型（LLM）能力，将复杂的招聘流程自动化，核心功能包括：通过自然语言对话生成结构化职位描述（JD）、批量简历解析与智能匹配评分、以及自动化生成面试题和录用/拒信。

本项目采用 **前后端分离** 架构，前端注重交互体验（"Bauhaus" 极简风格 + 动态视效），后端基于 Python/FastAPI 提供强大的 AI 处理能力。

2. 系统架构 (System Architecture)

2.1 技术栈 (Tech Stack)

- **Frontend**:
  - **Framework**: React 18 + Vite
  - **Styling**: TailwindCSS (原子化 CSS)
  - **Motion**: Framer Motion (用于复杂 UI 动画和页面流转)
  - **Icons**: Lucide React
- **Backend**:
  - **Framework**: FastAPI (高性能 Python Web 框架)
  - **Server**: Uvicorn (ASGI http 服务器)
- **AI / LLM Layer**:
  - **Model Provider**: ModelScope (阿里魔搭) / OpenAI API 兼容接口
  - **Models**: 接入 GLM, Qwen 等模型进行自然语言理解与生成。
- **Deployment**:
  - Docker 容器化部署，适配 ModelScope Spaces 环境。

2.2 核心模块 (Core Modules)

A. 职位画像生成引擎 (JD Generation Engine)

- **交互方式**: 采用多轮对话（Multi-turn Chat）形式。
- **技术实现**:
  - 用户在前端 Landing Page 输入模糊需求（如“招一个 Python 后端专家”）。

- 后端 `/api/chat` 接口维护对话上下文（`UserState.chat_history`），利用 LLM 逐步引导用户补充核心技能、经验要求、薪资范围等缺失信息。
- 当前端检测到信息收集完备，调用 `/api/generate_jd`，将收集到的非结构化信息由 LLM 转换为结构化的 `JobDefinition` JSON 对象（包含 `title`，`required_skills`，`experience_level` 等字段）。

B. 简历智能解析与匹配 (Resume Parsing & Matching)

- **数据摄入**: 支持 ZIP 压缩包或单文件（PDF/TXT/MD）上传。
- **解析流程**:
  - 后端 `/api/upload_resumes` 接收文件。
  - **Text Extraction**: 使用 `PyMuPDF (fitz)` 提取 PDF 文本。
  - **Batch Processing**: 实现了 `pdf_queue` 批处理机制，将多个简历文本合并为一个 Prompt 发送给 LLM 进行结构化提取（提取姓名、技能、工作年限等），提高处理效率并降低 API 调用频率。
- **智能评分 (Scoring)**:
  - 调用 `/api/analyze_resumes`。
  - **CoT (Chain of Thought)**: LLM 依据已生成的 `JobDefinition` 对每份简历进行深度比对，生成评分（0-100）、匹配摘要以及具体的“证据链”（Evidence，即简历中的具体原话佐证）。

C. 决策辅助系统 (Decision Support System)

- **可视化呈现**: 前端通过 Spotlight Card 动效展示候选人，包含评分大字报、AI 推荐理由及折叠的证据详情。
- **后续行动 (Action Generation)**:
  - 支持三种决策动作：`Interview`（面试），`Offer`（录用），`Reject`（婉拒）。
  - 后端 `/api/generate_action` 根据候选人画像和职位信息，动态生成个性化的邮件草稿或结构化面试题库。

3. 数据流转 (Data Flow)

1. **User Input** -> `Chat Interface` -> **LLM Reasoning** -> `Clarification Questions`
2. **Confirmed Intent** -> `JD Generator` -> **Structured JD (JSON)**
3. **Resume Files** -> `File Uploader` -> `Text Extractor` -> `Batch Processor` -> **Structured Resumes**
4. **Matching**: `Structure JD` + `Structured Resumes` -> `Ranking Engine` -> **Ranked Candidates List**
5. **Action**: `Candidate Context` + `Decision Type` -> `Content Generator` -> **Draft Email/Questions**

4. 关键技术亮点 (Key Features)

- **会话状态管理**: 后端使用内存字典 `SESSIONS` 管理用户状态，实现了简易的 Session 机制，确保多轮对话上下文的一致性。
- **流式体验模拟**: 虽然底层部分 API 为同步/异步响应，但前端通过精心设计的动画和模拟日志输出（`AI_PROCESS_LOG`），给用户提供了极具科技感和透明度的“AI 思考中”体验。

- **鲁棒的解析设计**: 针对中文 PDF 解析常见的乱码或格式问题，集成了特定的预处理逻辑；针对 ZIP 上传，支持自动解压与遍历。

5. 未来扩展方向 (Future Considerations)

- **向量数据库集成**: 目前简历匹配基于 Context Window 直接输入，未来可引入 RAG（检索增强生成）技术，利用向量数据库（如 Faiss/Milvus）处理海量人才库的初步召回。
- **多模型协同**: 针对不同任务（如简单的文本提取 vs 复杂的推理评分）动态路由到不同规模的模型，以平衡成本与效果。